

Finalisierungsphase

Dokumentation

Internationale Hochschule Duales MyStudium

Studiengang Informatik

3.Semester

Projekt: Data-Mart-Erstellung in SQL

DLBDSPBDM01_D

Eingereicht von

Brian Pudelko

Matrikelnummer: 4252311

brian.pudelko@iu-study.org

Inhaltsverzeichnis

1. Systemüberblick und Kernfunktionalitäten.....	1
1.1 Anwendungsfall und Systemarchitektur	1
1.2 Datenintegrität und Schutzmechanismen	1
2. Technische Umsetzung und Performance	1
2.1 Datenbankumgebung und relationale Umsetzung	2
2.2 Abfragen und Aggregationen	2
2.3 Performanceanalyse und Indexierung	2
3. Metadaten und Systemstatistik	2
3.1 Systemkennzahlen	3
3.2 Fazit	3

1. Systemüberblick und Kernfunktionalitäten

1.1 Anwendungsfall und Systemarchitektur

BookLoop bildet eine Plattform für die gemeinschaftliche Ausleihe von Büchern als relationale Datenbank ab. Benutzer können Bücher als Angebote zur Verfügung stellen, verfügbare Angebote reservieren und Bücher ausleihen. Zusätzlich verwaltet das System Verfügbarkeiten, Standorte und Bewertungen zu vorhandenen Ausleihvorgängen.

Das Datenmodell deckt den Ablauf von der Bereitstellung eines Buches bis zur Ausleihe und anschließenden Bewertung ab. Als zentrales Element habe ich das Angebot gewählt. Es verbindet einen Benutzer als Anbieter mit einem Buch und dessen Zustand sowie optional mit einem Standort. Reservierungen und Ausleihen beziehen sich anschließend auf dieses Angebot. Bewertungen ordne ich direkt einem vorhandenen Ausleihvorgang zu.

Stammdaten wie Rollen, Autoren, Verlage, Genres, Sprachen und Buchzustände liegen in eigenen Tabellen. Insgesamt besteht das relationale Datenmodell aus 14 miteinander verknüpften Tabellen. Die Aufteilung trennt die verschiedenen Informationen voneinander und verbindet sie über die jeweiligen Beziehungen.

1.2 Datenintegrität und Schutzmechanismen

Die Datenintegrität sichere ich durch verschiedene Mechanismen direkt auf Datenbankebene ab. Primärschlüssel identifizieren jeden Datensatz eindeutig. Fremdschlüssel bilden die Beziehungen zwischen den Tabellen ab und verhindern Verweise auf nicht vorhandene Datensätze. NOT NULL- und UNIQUE-Constraints stellen notwendige Angaben und die Eindeutigkeit bestimmter Attribute sicher. Ein Beispiel dafür ist die E-Mail-Adresse eines Benutzers.

Für fachliche Wertebereiche nutze ich zusätzliche CHECK-Constraints. Eine Bewertung darf beispielsweise nur zwischen einem und fünf Sternen liegen. Die maximale Leihdauer eines Angebots muss größer als null sein und das Ende eines Verfügbarkeitszeitraums nach dessen Beginn liegen. Die fest definierten Statuswerte bilde ich mit den PostgreSQL-ENUM-Datentypen anbotstatus, reservierungsstatus und ausleihstatus ab.

Bei Reservierungen übernimmt ein Trigger eine zusätzliche fachliche Prüfung. Er verhindert das Anlegen einer Reservierung, wenn das betreffende Angebot nicht den Status verfügbar besitzt. Die Wirksamkeit meiner Schutzmechanismen habe ich durch gezielte Positiv- und Negativtests im Detail überprüft. In meinen Testfällen provoziere ich bewusst Verletzungen von Primär-, Fremd- und UNIQUE-Schlüsseln, verwende ungültige ENUM-Werte oder gebe unzulässige Zeiträume ein, um die Reaktion von PostgreSQL zu überprüfen.

2. Technische Umsetzung und Performance

2.1 Datenbankumgebung und relationale Umsetzung

Für die technische Umsetzung von BookLoop nutze ich PostgreSQL 18 als relationales Datenbankmanagementsystem und pgAdmin 4, Version 9.11, für die Administration und Ausführung der SQL-Anweisungen. Alle Datenbankobjekte liegen im PostgreSQL-Schema public.

Die Implementierung habe ich auf drei SQL-Dateien aufgeteilt. 01_schema.sql enthält das vollständige Datenbankschema mit Tabellen, Datentypen, Constraints, Trigger und Indizes. 02_testdaten.sql befüllt die Tabellen mit den vorgesehenen Testdaten. Die Abfragen und Tests zur Überprüfung der Funktionen und Integritätsbedingungen befinden sich in 03_testfaelle.sql.

Die Beziehungen aus dem konzeptionellen Datenmodell habe ich über Fremdschlüssel in das relationale Schema übertragen. So lassen sich auch Informationen über mehrere Tabellen hinweg nachvollziehen. Über Benutzer → Ausleihe → Angebot → Buch lässt sich beispielsweise ermitteln, welcher Benutzer welches angebotene Buch ausgeliehen hat. Die Aufteilung in fachlich getrennte Tabellen vermeidet unnötige Redundanzen und hält die einzelnen Informationen klar voneinander getrennt.

2.2 Abfragen und Aggregationen

Für die Überprüfung der relationalen Beziehungen nutze ich verschiedene Abfragen, die Informationen aus mehreren Tabellen zusammenführen. Je nach beteiligten Attributen kommen JOIN-Operationen mit USING oder ON zum Einsatz. Auf diese Weise prüfe ich nicht nur einzelne Tabellen, sondern auch die im Datenmodell vorgesehenen Zusammenhänge.

Zusätzlich habe ich mehrere Aggregationsabfragen umgesetzt. Eine davon zählt die Ausleihvorgänge je Benutzer und zeigt damit die aktivsten Nutzer im vorhandenen Testdatenbestand. Eine weitere gruppiert die Ausleihen nach Genre und zeigt die am häufigsten ausgeliehenen Genres. Für abgeschlossene Ausleihvorgänge ermittle ich außerdem die durchschnittliche sowie die kürzeste und längste Ausleihdauer.

Mit diesen Abfragen überprüfe ich die Beziehungen des Datenmodells und zeige gleichzeitig, dass sich die gespeicherten Daten für übergreifende Auswertungen kombinieren und zusammenfassen lassen.

2.3 Performanceanalyse und Indexierung

Für die Performanceanalyse habe ich zunächst mit der PostgreSQL-Systemansicht pg_indexes geprüft, welche Indizes bereits im Schema vorhanden sind. Die Abfrage ergab 23 Indizes, die insbesondere durch Primärschlüssel und UNIQUE-Constraints entstanden sind.

PostgreSQL legt für gewöhnliche Fremdschlüssel nicht automatisch entsprechende Indizes an. Deshalb habe ich die Beziehungen betrachtet, die in den Abfragen regelmäßig für Verknüpfungen verwendet werden. Auf dieser Grundlage ergänzte ich vier Indizes für `angebot.buch_id`, `angebot.anbieter_id`, `reservierung.angebot_id` und `ausleihe.angebot_id`.

Nach dieser Erweiterung enthält das Schema insgesamt 27 Indizes. Bei den aktuell zehn Testdatensätzen pro Tabelle wäre ein gemessener Geschwindigkeitsvorteil wenig aussagekräftig. Die zusätzlichen Indizes sind daher vor allem für einen zukünftig größeren Datenbestand vorgesehen.

3. Metadaten und Systemstatistik

3.1 Systemkennzahlen

Die BookLoop-Datenbank umfasst 14 Tabellen im Schema `public` und insgesamt 27 Indizes. 23 dieser Indizes entstanden durch die Definition von Primärschlüsseln und `UNIQUE`-Constraints. Vier weitere habe ich gezielt für ausgewählte Fremdschlüsselbeziehungen ergänzt.

Für die Funktionsprüfung enthält jede der 14 Tabellen 10 Testdatensätze. Insgesamt umfasst der Testdatenbestand damit 140 Datensätze. Die Daten verteilen sich auf Rolle, Benutzer, Standort, Autor, Verlag, Genre, Sprache, Buch, Zustand, Angebot, Verfügbarkeit, Reservierung, Ausleihe und Bewertung. Damit kann ich die Beziehungen und Funktionen aller Entitäten mit vorhandenen Daten testen.

Die physische Größe der Datenbank habe ich direkt in PostgreSQL mit `pg_database_size()` ausgelesen. Zum Zeitpunkt der Messung beträgt das Datenbankvolumen 9062 kB. Neben der Anzahl der Tabellen, Datensätze und Indizes liefert dieser Wert eine zusätzliche Kennzahl zum tatsächlichen Speicherbedarf der implementierten Datenbank.

3.2 Fazit

Mit BookLoop habe ich das konzeptionelle Datenmodell in eine funktionsfähige relationale PostgreSQL-Datenbank überführt. Beziehungen, Constraints und der Trigger sichern die Datenintegrität ab, während die Testfälle die vorgesehenen Funktionen überprüfen. Die ergänzten Indizes berücksichtigen zusätzlich einen zukünftig wachsenden Datenbestand. Damit bildet die Datenbank die vorgesehenen Abläufe von BookLoop konsistent ab.